Phase 4 Structured Note: Depth-First Search (DFS)

1. Current Learning Position

You are now in **Phase 4** of the graph curriculum.

Before this phase, you already learned three important things:

Previous phase	What you learned	Why it matters for DFS
Phase 1	Graph vocabulary: vertex, edge, path, cycle, directed graph, undirected graph	DFS uses these words constantly.
Phase 2	Graph representation: adjacency matrix and adjacency list	DFS needs the graph to be stored in memory before it can run.
Phase 3	BFS: level-by-level traversal using a queue and visited array	DFS also uses a visited array, but it explores in a different way.

Now you are ready for **Depth-First Search**, usually written as **DFS**.

Phase 4 answers this question:

> Once a graph is stored in memory, how do we visit its vertices by going deep first?

---

2. What Is Covered in Phase 4?

In this phase, you will learn:

1. What DFS means
2. DFS intuition
3. DFS compared with BFS
4. What DFS needs
5. The visited array in DFS
6. Why DFS uses recursion or a stack
7. The system stack
8. Backtracking
9. DFS algorithm steps
10. DFS pseudocode
11. DFS using an adjacency matrix
12. DFS using an adjacency list
13. Complete C++ DFS program
14. DFS dry run from vertex 1

15. DFS spanning tree
16. Tree edges
17. Back edges
18. Multiple valid DFS orders
19. Infinite recursion prevention
20. DFS time complexity
21. DFS space complexity
22. Common beginner mistakes
23. Practice questions
24. Final checklist

---

3. What Does DFS Mean?

DFS stands for:

Depth-First Search

Simple meaning:

> DFS starts from one vertex, follows one path as deeply as possible, and only comes back when it cannot go farther.

The word **depth** means going deep.

So DFS does not try to visit all nearby vertices first. Instead, it chooses one neighbor and keeps going deeper and deeper.

---

4. Simple DFS Intuition

Think of DFS like exploring a maze.

You enter the maze and choose one hallway.

You keep walking down that hallway until you reach a dead end.

Then you go back to the last place where there was another path.

Then you try that other path.

That is DFS.

Simple memory idea:

```
DFS = choose one path and go as far as possible
```

---

5. Real-World Analogy

Imagine you are exploring roads from your house.

DFS behaves like this:

1. Start at your house.
2. Choose one road.
3. Keep following roads forward.
4. If you reach a dead end, go back.
5. Try another road.

DFS is not trying to explore all nearby roads first.

It wants to go deep along one route before trying another route.

---

6. BFS vs DFS

You already learned BFS in Phase 3.

BFS and DFS are both graph traversal algorithms, but they explore differently.

Algorithm	Full name	Main idea	Main data structure
BFS	Breadth-First Search	Visit level by level	Queue
DFS	Depth-First Search	Go deep first, then backtrack	Recursion or stack

BFS idea

BFS says:

> Visit all nearby vertices first.

DFS idea

DFS says:

> Pick one unvisited neighbor and go deep immediately.

---

7. Example: BFS vs DFS

Suppose we have this graph:

```
1 ---- 2 ---- 4
|      |
3      5
```

Starting from vertex `1`:

Possible BFS order

```
1 2 3 4 5
```

BFS visits nearby vertices first.

It visits `2` and `3` before going deeper to `4` and `5`.

Possible DFS order

```
1 2 4 5 3
```

DFS goes from `1` to `2`, then deeper to `4`, then backtracks and continues.

Important idea:

> BFS spreads outward. DFS dives inward.

---

8. What DFS Needs

DFS needs three main things:

Item	Purpose
Graph representation	Stores the vertices and edges
Visited array	Remembers which vertices have already been seen
Recursion or stack	Remembers where DFS should return after going deep

The graph representation may be:

- adjacency matrix
- adjacency list

The visited array prevents repeated visits.

Recursion or a stack controls the deep-first behavior.

---

9. The Visited Array in DFS

A visited array stores whether each vertex has already been visited.

Example:

```
vector<int> visited(n + 1, 0);
```

Here:

```
0 means not visited
1 means visited
```

If the graph has vertices `1` to `6`, then at the beginning:

```
visited[1] = 0
visited[2] = 0
visited[3] = 0
visited[4] = 0
visited[5] = 0
visited[6] = 0
```

After DFS visits vertex `1`:

```
visited[1] = 1
```

---

10. Why DFS Needs a Visited Array

Graphs can contain cycles.

Example:

```
1 ---- 2
```

This graph is undirected.

That means:

```
1 can go to 2
2 can go back to 1
```

If DFS does not use a visited array, it may keep moving forever:

```
DFS(1)
DFS(2)
DFS(1)
DFS(2)
DFS(1)
DFS(2)
...
```

This is called **infinite recursion**.

The visited array prevents this.

Important rule:

> Mark a vertex as visited before calling DFS on its neighbors.

---

11. The Most Important DFS Rule

The most important DFS rule is:

```
Visit the vertex.
Mark it as visited.
Then explore its neighbors.
```

Correct order:

```
cout << start << " ";
visited[start] = 1;
```

Then DFS can safely go to the neighbors.

Wrong idea:

```
cout << start << " ";
// forgot visited[start] = 1
```

This can cause DFS to visit the same vertex again and again.

---

12. Recursion in DFS

DFS is commonly written using recursion.

A recursive function is a function that calls itself.

Example:

```
DFS(v);
```

This means:

> To continue DFS, call DFS again from neighbor `v`.

DFS uses recursion because recursion naturally supports going deep.

---

13. The System Stack

When a function calls another function, C++ remembers the old function using the **system stack**.

In DFS, this is very useful.

Example:

```
DFS(1) calls DFS(2)
DFS(2) calls DFS(4)
DFS(4) calls DFS(3)
```

The call stack looks like this:

```
DFS(3)   <- currently running
DFS(4)   <- paused
DFS(2)   <- paused
DFS(1)   <- paused
```

When `DFS(3)` finishes, the program returns to `DFS(4)`.

Then `DFS(4)` continues checking its remaining neighbors.

Simple meaning:

> The stack remembers where DFS must return after going deep.

---

14. Stack Behavior in Simple Words

A stack follows this rule:

```
Last In, First Out
```

This is also called **LIFO**.

The most recent function call is the first one to finish and return.

That is exactly what DFS needs.

Memory trick:

```
DFS = recursion = stack = deep first
BFS = queue = level first
```

---

15. What Is Backtracking?

Backtracking means:

> When DFS reaches a vertex with no unvisited neighbors, it returns to the previous vertex and continues from there.

Example:

```
1 -> 2 -> 4 -> 3
```

Suppose vertex `3` has no new neighbor.

DFS cannot go deeper from `3`.

So DFS goes back to `4`.

Then DFS checks whether `4` has another unvisited neighbor.

Simple meaning:

```
Go deep.
Hit a dead end.
Go back.
Try another path.
```

---

16. DFS in Simple Steps

Suppose DFS starts at vertex `start`.

The steps are:

1. Visit `start`.
2. Mark `start` as visited.
3. Look at each neighbor of `start`.
4. If a neighbor is not visited, immediately call DFS on that neighbor.
5. When that neighbor's DFS finishes, come back.
6. Continue checking the remaining neighbors.
7. Stop when there are no more unvisited neighbors.

Simple version:

```
Visit this vertex.
Mark it visited.
For each unvisited neighbor:
    go there immediately.
When stuck:
    return to the previous vertex.
```

---

17. DFS Pseudocode

```
DFS(u):
    mark u as visited
    print u

    for each neighbor v of u:
        if v is not visited:
            DFS(v)
```

The important line is:

```
DFS(v)
```

This line makes DFS go deeper immediately.

---

18. DFS Using an Adjacency Matrix

In an adjacency matrix:

```
G[u][v] == 1
```

means:

```
There is an edge from u to v.
```

To find the neighbors of vertex `u`, DFS scans a full row of the matrix.

Example:

```
for (int v = 1; v <= n; v++) {
    if (G[u][v] == 1 && visited[v] == 0) {
        DFS(G, visited, v, n);
    }
}
```

This checks every possible vertex `v`.

If `G[u][v] == 1`, then `v` is a neighbor of `u`.

If `visited[v] == 0`, then DFS has not visited `v` yet.

So DFS calls itself on `v`.

---

19. Complete C++ DFS Program Using an Adjacency Matrix

```
#include <iostream>
#include <vector>
using namespace std;

void DFS(const vector<vector<int>>& G, vector<int>& visited, int start, int n) {
    if (visited[start] == 0) {
        cout << start << " ";
        visited[start] = 1; // very important: mark before going deeper

        for (int v = 1; v <= n; v++) {
            if (G[start][v] == 1 && visited[v] == 0) {
                DFS(G, visited, v, n);
            }
        }
    }
}

int main() {
    int n = 6;

    vector<vector<int>> G(n + 1, vector<int>(n + 1, 0));

    auto addUndirectedEdge = [&](int u, int v) {
        G[u][v] = 1;
        G[v][u] = 1;
    };

    addUndirectedEdge(1, 2);
```

```
    addUndirectedEdge(1, 3);
    addUndirectedEdge(2, 4);
    addUndirectedEdge(3, 4);
    addUndirectedEdge(4, 5);
    addUndirectedEdge(4, 6);

    vector<int> visited(n + 1, 0);

    cout << "DFS from vertex 1: ";
    DFS(G, visited, 1, n);

    return 0;
}
```

Expected output:

```
DFS from vertex 1: 1 2 4 3 5 6
```

---

20. Code Explanation in Simple Words

Include libraries

```
#include <iostream>
#include <vector>
```

These allow us to use:

- `cout`
- `vector`

DFS function header

```
void DFS(const vector<vector<int>>& G, vector<int>& visited, int start, int n)
```

This function receives:

Parameter	Meaning
`G`	The graph stored as an adjacency matrix
`visited`	The array that remembers visited vertices

Parameter	Meaning
`start`	The current vertex DFS is visiting
`n`	Number of vertices

Check whether current vertex is unvisited

```
if (visited[start] == 0)
```

This means:

> Only process this vertex if it has not been visited yet.

Print current vertex

```
cout << start << " ";
```

This displays the vertex when DFS first visits it.

Mark current vertex

```
visited[start] = 1;
```

This prevents the same vertex from being visited again.

Check all possible neighbors

```
for (int v = 1; v <= n; v++)
```

Because we are using a matrix, DFS checks all possible vertices.

Recursive call

```
DFS(G, visited, v, n);
```

This means:

> Go deeper from neighbor `v`.

---

21. Graph Used in the Program

The program creates this graph:

```
1 ---- 2
|      |
3 ---- 4 ---- 5
       |
       6
```

Edges:

```
1--2
1--3
2--4
3--4
4--5
4--6
```

Because the graph is undirected, every edge is stored in both directions.

Example:

```
G[u][v] = 1;
G[v][u] = 1;
```

---

22. DFS Dry Run from Vertex 1

DFS starts at vertex `1`.

The graph is:

```
1 ---- 2
|      |
3 ---- 4 ---- 5
       |
       6
```

Assume the matrix checks neighbors from small number to large number.

So from a vertex, DFS checks:

```
1, 2, 3, 4, 5, 6
```

---

23. Step-by-Step DFS Dry Run

Step	What happens	Output so far
1	Start at `1`. Visit it and mark it visited.	`1`
2	From `1`, check neighbors. The smallest unvisited neighbor is `2`. Go to `2`.	`1 2`
3	From `2`, neighbor `1` is already visited. Next unvisited neighbor is `4`. Go to `4`.	`1 2 4`
4	From `4`, neighbor `2` is already visited. Next unvisited neighbor is `3`. Go to `3`.	`1 2 4 3`
5	From `3`, neighbors `1` and `4` are already visited. Dead end. Backtrack to `4`.	`1 2 4 3`
6	Back at `4`, next unvisited neighbor is `5`. Go to `5`.	`1 2 4 3 5`
7	From `5`, no new neighbor exists. Backtrack to `4`.	`1 2 4 3 5`
8	Back at `4`, next unvisited neighbor is `6`. Go to `6`.	`1 2 4 3 5 6`
9	From `6`, no new neighbor exists. DFS finishes.	`1 2 4 3 5 6`

Final DFS order:

```
1 2 4 3 5 6
```

---

24. Understanding the Recursion Stack During the Dry Run

DFS calls happen like this:

```
DFS(1)
  DFS(2)
    DFS(4)
      DFS(3)
```

At this moment, DFS is deep at vertex `3`.

When `DFS(3)` finishes, it returns to `DFS(4)`.

Then `DFS(4)` continues and calls:

```
DFS(5)
```

After `DFS(5)` finishes, it returns again to `DFS(4)`.

Then DFS calls:

```
DFS(6)
```

This is backtracking.

---

25. DFS Using an Adjacency List

DFS can also use an adjacency list.

An adjacency list stores only real neighbors.

Example graph:

```
1 ---- 2
|      |
3 ---- 4
```

Adjacency list:

```
1: 2, 3
2: 1, 4
3: 1, 4
4: 2, 3
```

DFS using an adjacency list checks only the stored neighbors.

This is usually more efficient for sparse graphs.

---

26. DFS C++ Program Using an Adjacency List

```cpp
#include <iostream>
#include <vector>
using namespace std;

void DFS(const vector<vector<int>>& adj, vector<int>& visited, int u) {
    visited[u] = 1;
    cout << u << " ";

    for (int v : adj[u]) {
        if (visited[v] == 0) {
            DFS(adj, visited, v);
        }
    }
}

int main() {
    int n = 6;
    vector<vector<int>> adj(n + 1);

    auto addEdge = [&](int u, int v) {
        adj[u].push_back(v);
        adj[v].push_back(u);
    };

    addEdge(1, 2);
    addEdge(1, 3);
    addEdge(2, 4);
    addEdge(3, 4);
    addEdge(4, 5);
    addEdge(4, 6);

    vector<int> visited(n + 1, 0);

    cout << "DFS from vertex 1: ";
    DFS(adj, visited, 1);

    return 0;
}
```

Possible output:

```
DFS from vertex 1: 1 2 4 3 5 6
```

The exact output depends on the order of neighbors in the adjacency list.

---

27. Matrix DFS vs List DFS

Feature	Matrix DFS	List DFS
Graph storage	2D table	List of neighbors
Neighbor checking	Checks every possible vertex	Checks only actual neighbors
Time complexity	`O(N^2)`	`O(V + E)`
Good for	Dense graphs	Sparse graphs
Beginner difficulty	Very direct	Usually more efficient

Simple rule:

```
Matrix DFS scans all possible neighbors.
List DFS scans only real neighbors.
```

---

28. DFS Spanning Tree

When DFS runs, it can create a tree-like structure called a **DFS spanning tree**.

Simple meaning:

> A DFS spanning tree shows the edges DFS used to discover new vertices.

Example:

If DFS visits:

```
1 -> 2 -> 4 -> 3
```

Then these discovery edges are tree edges:

```
1--2
2--4
4--3
```

These edges helped DFS reach a new vertex for the first time.

---

29. Tree Edges

A **tree edge** is an edge used to discover a new vertex.

Example:

```
1 discovers 2
2 discovers 4
4 discovers 3
```

The edges are:

```
1--2
2--4
4--3
```

These are tree edges.

Simple meaning:

> A tree edge is a discovery edge.

---

30. Back Edges

A **back edge** is an edge that points to a vertex that was already visited.

In DFS, a back edge often connects a vertex back to an ancestor in the DFS tree.

Example:

Suppose DFS already visited:

```
1 -> 2 -> 4
```

If DFS sees an edge from `4` back to `1`, then `1` is already visited.

That edge is not used to discover a new vertex.

So it can be treated as a back edge.

Simple meaning:

> A back edge is an extra edge that goes back to an already visited vertex.

---

31. Why Back Edges Matter

Back edges are important because they can show that a graph has a cycle.

Example:

```
1 ---- 2
|      |
4 ---- 3
```

This graph has a cycle:

```
1 -> 2 -> 3 -> 4 -> 1
```

During DFS, if an edge goes back to an already visited ancestor, that can indicate a cycle.

For beginners, remember this simple idea:

```
Tree edge = discovers a new vertex
Back edge = connects to an already visited vertex
```

---

32. Multiple Valid DFS Orders

DFS does not always have one single correct output.

The DFS order depends on:

1. The starting vertex
2. The order in which neighbors are checked
3. The graph representation
4. The order edges were inserted into an adjacency list

Example:

If DFS starts from `1` and checks smaller neighbors first, it may output:

```
1 2 4 3 5 6
```

If DFS checks another neighbor first, it may output:

```
1 3 4 2 5 6
```

Both can be valid if DFS follows the deep-first rule.

Important beginner rule:

> Do not memorize only one DFS output. Understand the rule that produced the output.

---

33. DFS on Directed Graphs

DFS also works on directed graphs.

The difference is:

> In a directed graph, DFS must follow the arrow direction.

Example:

```
1 ---> 2 ---> 3
```

DFS from `1` can visit:

```
1 2 3
```

But DFS from `3` visits only:

```
3
```

Why?

Because there is no arrow from `3` back to `2` or `1`.

Important rule:

```
In directed graphs, DFS can only move in the direction of the edge.
```

---

34. DFS on Undirected Graphs

In an undirected graph, edges work both ways.

Example:

```
1 ---- 2 ---- 3
```

DFS from `1` can visit:

```
1 2 3
```

DFS from `3` can visit:

```
3 2 1
```

Because the edges can be traveled in both directions.

Important rule:

```
For undirected graphs, store both directions.
```

Example:

```
adj[u].push_back(v);
adj[v].push_back(u);
```

---

35. DFS for Disconnected Graphs

If a graph is disconnected, one DFS may not visit all vertices.

Example:

```
1 ---- 2     3 ---- 4     5
```

There are three connected components:

```
{1, 2}
{3, 4}
{5}
```

If DFS starts from `1`, it visits only:

```
1 2
```

It cannot reach `3`, `4`, or `5`.

To visit the whole graph, run DFS from every unvisited vertex.

---

36. DFS for All Components: C++ Idea

```cpp
for (int i = 1; i <= n; i++) {
    if (visited[i] == 0) {
        DFS(adj, visited, i);
    }
}
```

Simple meaning:

1. Check every vertex.
2. If it is not visited, it starts a new component.
3. Run DFS from it.

This can also count connected components.

---

37. DFS Time Complexity

DFS time complexity depends on the graph representation.

37.1 DFS with Adjacency Matrix

In an adjacency matrix, DFS may scan a full row for each vertex.

If there are `N` vertices, each row has `N` cells.

In the worst case, DFS scans `N` rows.

So the time complexity is:

```
O(N^2)
```

Why?

```
N vertices × N checks per vertex = N^2 checks
```

37.2 DFS with Adjacency List

In an adjacency list, DFS checks only real neighbors.

Each vertex is visited once.

Each edge is considered through the neighbor lists.

So the time complexity is:

```
O(V + E)
```

Where:

```
V = number of vertices
E = number of edges
```

---

38. DFS Space Complexity

DFS uses extra memory for:

1. The visited array

2. The recursion stack

The visited array stores one value for each vertex.

So it uses:

```
O(V)
```

The recursion stack can also grow up to `V` calls in the worst case.

So DFS extra space complexity is:

```
O(V)
```

If the notes use `N` instead of `V`, this may also be written as:

```
O(N)
```

Important note:

> This is extra memory used by DFS. The graph itself also uses memory depending on whether it is stored as a matrix or list.

---

39. Infinite Recursion Warning

One of the biggest DFS mistakes is forgetting to mark a vertex as visited before recursion.

Wrong idea:

```cpp
void DFS(int u) {
    cout << u << " ";

    for (int v : adj[u]) {
        if (visited[v] == 0) {
            DFS(v);
        }
    }
}
```

Problem:

`u` was never marked visited.

So another vertex may call DFS on `u` again.

Correct version:

```
void DFS(int u) {
    visited[u] = 1;
    cout << u << " ";

    for (int v : adj[u]) {
        if (visited[v] == 0) {
            DFS(v);
        }
    }
}
```

Golden rule:

```
Mark first, then recurse.
```

---

40. Common Beginner Mistakes

Mistake 1: Forgetting the visited array

Wrong idea:

```
DFS can just keep following edges.
```

Problem:

Graphs can contain cycles.

Correct idea:

```
Use visited[] to avoid repeating vertices.
```

Mistake 2: Forgetting to mark visited before recursion

Wrong:

```
DFS(v);
visited[v] = 1;
```

Better:

```
visited[v] = 1;
DFS(v);
```

Or inside the DFS function:

```
visited[u] = 1;
```

before exploring neighbors.

Mistake 3: Expecting DFS to visit all direct neighbors first

That is BFS behavior.

DFS does not visit all direct neighbors first.

DFS chooses one neighbor and goes deep.

Mistake 4: Confusing BFS and DFS data structures

BFS uses:

```
Queue
```

DFS uses:

```
Recursion or stack
```

Memory trick:

```
BFS = Queue
DFS = Stack or Recursion
```

Mistake 5: Expecting only one correct DFS order

DFS can have multiple valid orders.

The order depends on neighbor checking order.

Correct idea:

```
Check whether the traversal goes deep first.
```

Mistake 6: Not resetting visited before running DFS again

If you run DFS once, many vertices become visited.

If you want to run DFS again from scratch, reset the array.

Example:

```
vector<int> visited(n + 1, 0);
```

Mistake 7: Forgetting reverse edges in an undirected graph

Wrong for undirected graph:

```
adj[u].push_back(v);
```

Correct:

```
adj[u].push_back(v);
adj[v].push_back(u);
```

Mistake 8: Adding reverse edges in a directed graph

Wrong for directed graph:

```
adj[u].push_back(v);
adj[v].push_back(u);
```

Correct:

```
adj[u].push_back(v);
```

Mistake 9: Losing track of recursion

Beginner problem:

```
I do not know where DFS returns after a recursive call finishes.
```

Correct idea:

```
DFS returns to the previous paused function call.
```

The system stack remembers where to continue.

---

41. Phase 4 Summary Table

Concept	Simple meaning
DFS	Depth-First Search
Graph traversal	Visiting graph vertices in an organized way
Depth first	Go as deep as possible before backtracking
Visited array	Prevents repeated visits
Recursion	A function calling itself
Stack	Structure that remembers paused DFS calls
System stack	The hidden stack C++ uses for function calls
Backtracking	Returning to a previous vertex after a dead end

Concept	Simple meaning
DFS spanning tree	Tree made from DFS discovery edges
Tree edge	Edge used to discover a new vertex
Back edge	Edge connecting to an already visited ancestor or earlier vertex
Multiple valid DFS orders	DFS output can change depending on neighbor order
Matrix DFS time	`O(N^2)`
List DFS time	`O(V + E)`
DFS extra space	`O(V)` or `O(N)`

---

42. BFS vs DFS Final Comparison

Feature	BFS	DFS
Full name	Breadth-First Search	Depth-First Search
Main behavior	Goes level by level	Goes deep first
Data structure	Queue	Stack or recursion
Memory rule	First discovered, first explored	Most recent path continues first
Good mental image	Water spreading outward	Maze exploration
Uses visited array	Yes	Yes
Can handle disconnected graphs	Yes, if run from every unvisited vertex	Yes, if run from every unvisited vertex

---

43. DFS Memory Tricks

Trick 1

```
DFS = Deep First Search
```

DFS goes deep before going wide.

Trick 2

```
DFS uses recursion.
Recursion uses stack.
Stack means last in, first out.
```

Trick 3

```
Visit, mark, go deep, backtrack.
```

Trick 4

```
Mark first, then recurse.
```

---

44. Mini Practice Questions

Question 1

What does DFS stand for?

Answer:

```
Depth-First Search
```

Question 2

What is the main idea of DFS?

Answer:

```
DFS goes as deep as possible along one path before backtracking.
```

Question 3

What data structure does DFS naturally use?

Answer:

```
Recursion or stack
```

Question 4

Why does DFS need a visited array?

Answer:

```
To avoid visiting the same vertex again and to prevent infinite recursion in
graphs with cycles.
```

Question 5

What is backtracking?

Answer:

```
Backtracking is returning to a previous vertex when the current path has no
unvisited neighbors.
```

Question 6

What is the time complexity of DFS using an adjacency matrix?

Answer:

```
O(N^2)
```

Question 7

What is the time complexity of DFS using an adjacency list?

Answer:

```
O(V + E)
```

Question 8

Can DFS have more than one valid output order?

Answer:

```
Yes. The order depends on the starting vertex and the order in which neighbors
are checked.
```

Question 9

What is a tree edge in DFS?

Answer:

```
A tree edge is an edge used to discover a new vertex.
```

Question 10

What is a back edge in DFS?

Answer:

```
A back edge is an edge that connects to a vertex that was already visited, often
an ancestor in the DFS tree.
```

---

45. Practice Dry Run

Try DFS on this graph:

```
1 ---- 2 ---- 5
|      |
3 ---- 4
```

Start from vertex `1`.

Assume neighbors are checked from small to large.

Step 1

Visit `1`.

```
output = 1
```

Step 2

From `1`, go to `2`.

```
output = 1 2
```

Step 3

From `2`, go to `4`.

```
output = 1 2 4
```

Step 4

From `4`, go to `3`.

```
output = 1 2 4 3
```

Step 5

From `3`, no new neighbor exists.

Backtrack to `4`.

Step 6

From `4`, no new neighbor exists.

Backtrack to `2`.

Step 7

From `2`, go to `5`.

```
output = 1 2 4 3 5
```

Final DFS order:

```
1 2 4 3 5
```

---

46. Final Phase 4 Checklist

You understand Phase 4 if you can explain these without looking:

- What DFS stands for
- What graph traversal means
- Why DFS is called depth-first
- How DFS differs from BFS
- Why DFS uses recursion or a stack
- What the system stack does
- What backtracking means
- Why DFS needs a visited array
- Why visited must be marked before recursion continues
- How DFS works using an adjacency matrix
- How DFS works using an adjacency list
- How to write DFS pseudocode
- How to dry run DFS by hand
- How to trace the recursion stack
- What a DFS spanning tree is
- What a tree edge is
- What a back edge is
- Why DFS can have multiple valid outputs
- How DFS behaves on directed graphs
- How DFS behaves on undirected graphs
- How to run DFS on disconnected graphs
- Why matrix DFS is `O(N^2)`
- Why list DFS is `O(V + E)`
- Why DFS extra space is `O(V)`
- How to avoid infinite recursion
- The most common beginner DFS mistakes

---

47. Final Phase 4 Takeaway

DFS is a graph traversal algorithm that explores deeply before backtracking.

It starts from one vertex, marks it as visited, then immediately moves to an unvisited neighbor.

It keeps going deeper until it reaches a dead end.

Then it backtracks to the previous vertex and tries another path.

The heart of DFS is:

```
Recursion or stack
```

The safety tool of DFS is:

```
Visited array
```

The most important DFS pattern is:

```
Visit the vertex.
Mark it visited.
For each unvisited neighbor:
    recursively run DFS.
Backtrack when no unvisited neighbor remains.
```

Final memory line:

```
DFS = deep branch-by-branch graph traversal using recursion and a visited array.
```

After Phase 4, you are ready for Phase 5: Disjoint Sets and graph problem solving.